Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

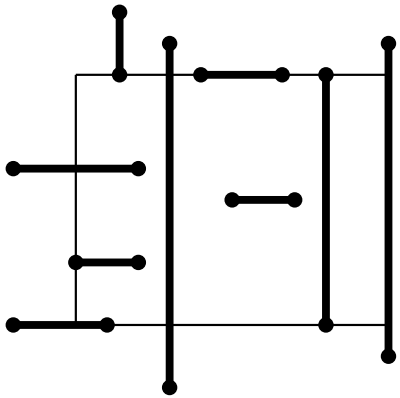
MCS 481 Lecture 30
Computational Geometry
Jan Verschelde, 2 November 2026

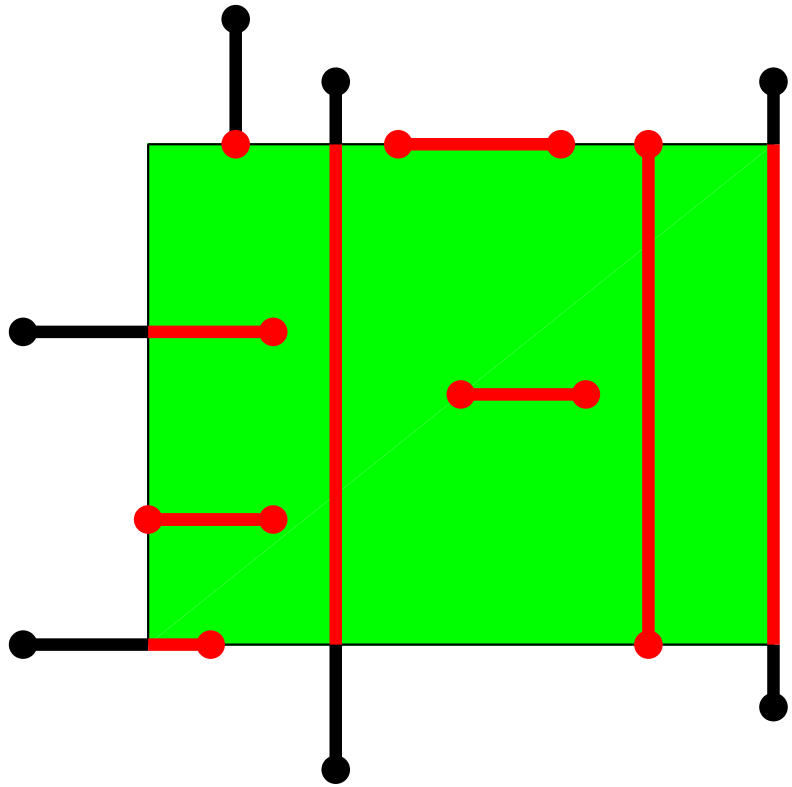
Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

windowing queries

Given a map, we zoom in on a window $\Rightarrow$ windowing query.





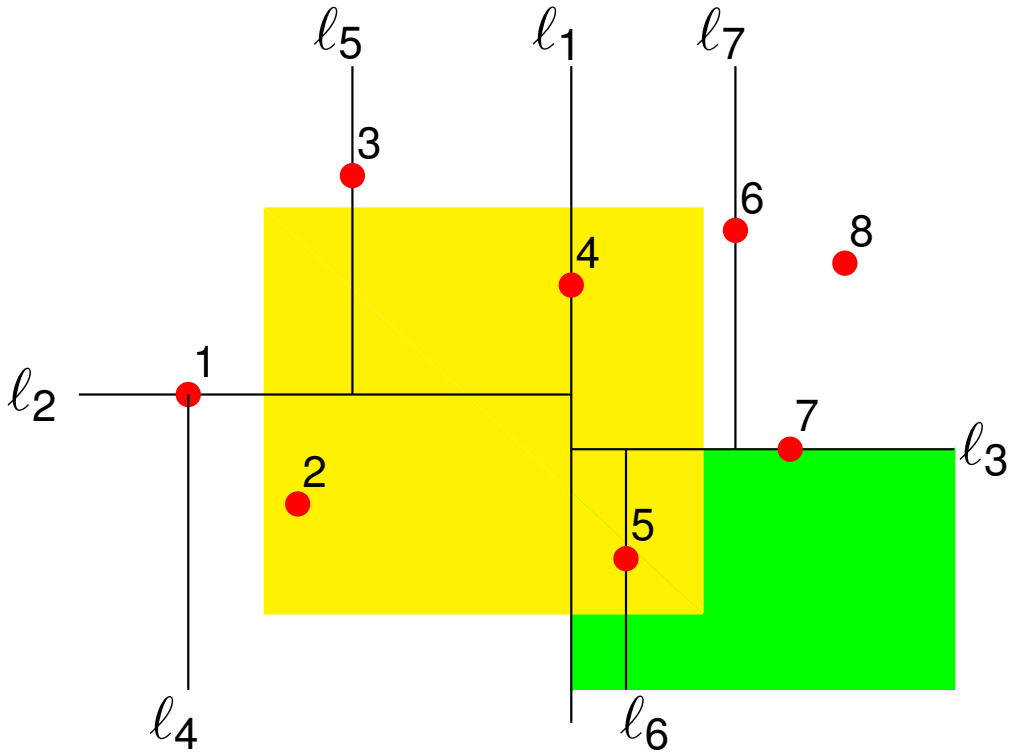
Problem: store data to execute windowing queries quickly.

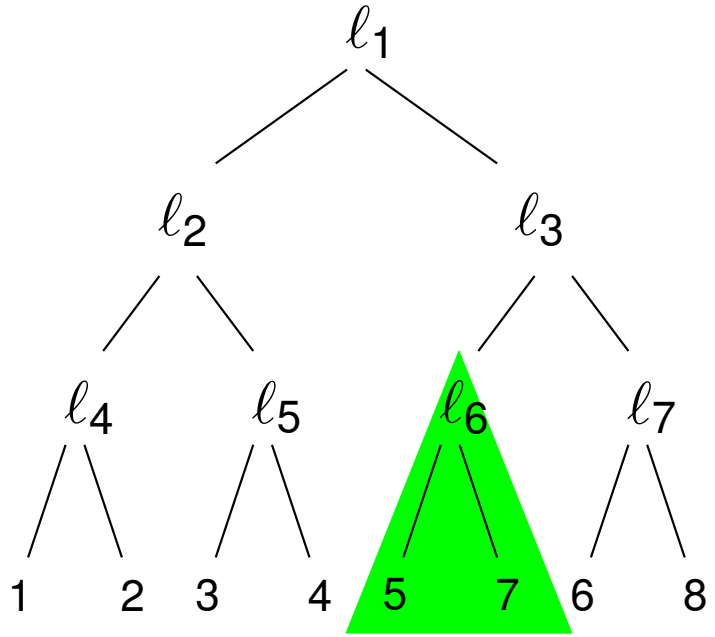
Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

orthogonal range queries — logarithmic cost

A range is a rectangle $[x : x'] \times [y : y']$. A range may be entirely contained in some region, but it may also overlap with several regions.

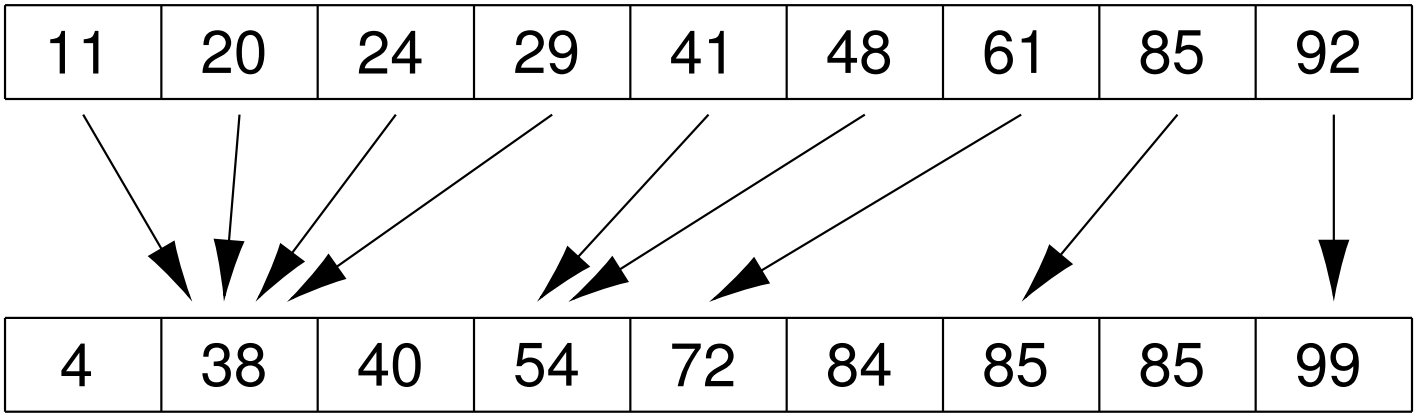




A range tree is a balanced binary tree on the x-coordinate, every node stores a balanced binary tree on the y-coordinate.

fractional cascading

We can save a binary search by linking sorted arrays.
Consider $A = [11, 20, 24, 29, 41, 48, 61, 85, 92]$
and $B = [4, 38, 40, 54, 72, 84, 85, 85, 99]$.



Every element x in A has a pointer to the first element in B which is either equal to x or larger than x.
Thanks to these pointers we save a binary search in B.

The query time is $O(\log(n) + k)$, in a 2D range tree of n points, where k is the number of reported points.

axis parallel query windows

Consider line segments parallel either to the x- or to the y-axis.

Definition (axis parallel query window)

An *axis parallel query window* W is determined by two intervals, $W = [x : x'] \times [y : y']$.

Applying the results from range queries, we have the following.

Lemma (cost of axis parallel query window)

Let S be a set of n axis parallel line segments and W be an axis parallel query window. Segments having at least one endpoint inside W can be reported in $O(\log(n) + k)$ time with a data structure using $O(n \log(n))$ storage and preprocessing time, where k is the number of reported segments.

Can we reduce the storage cost from $O(n \log(n))$ to $O(n)$?

Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

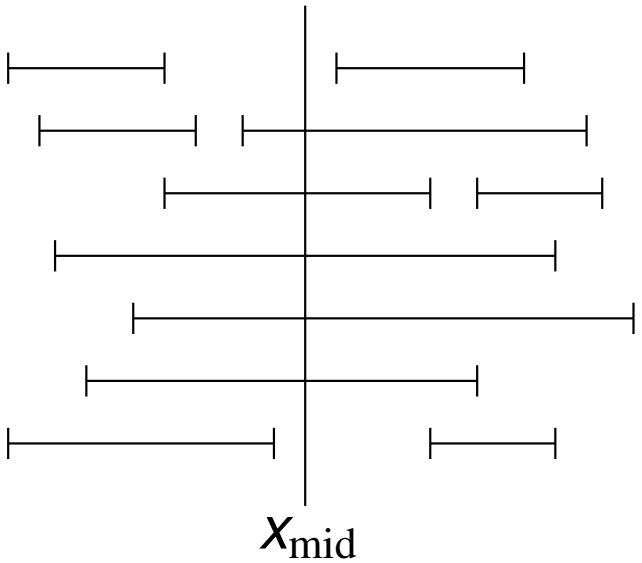
search horizontal line segments

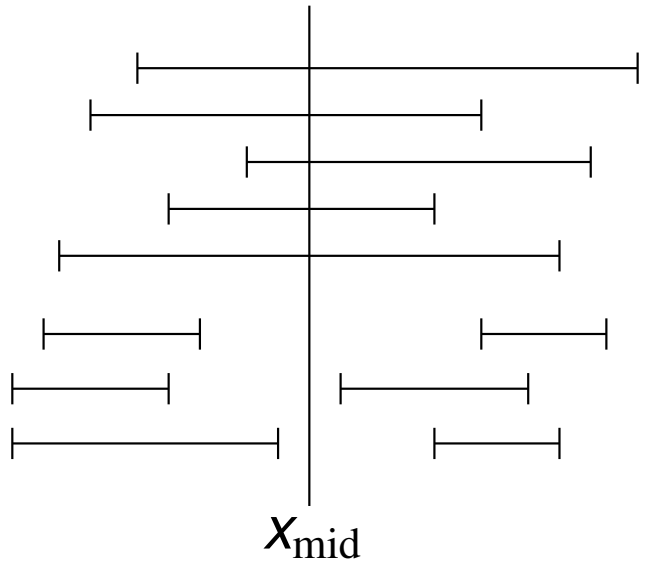
Problem Statement:

preprocess a set of n horizontal line segments such that segments intersected by a vertical line can be reported efficiently.

Input: $S = \{[x_1 : x_1'], [x_2 : x_2'], \ldots, [x_n : x_n']\}$, a set of n intervals.
x_{mid} is the median of the $2n$ end points so that

- at most half of the end points of S lies at the left of x_{mid}, and
- at most half of the end points of S lies at the right of x_{mid}.

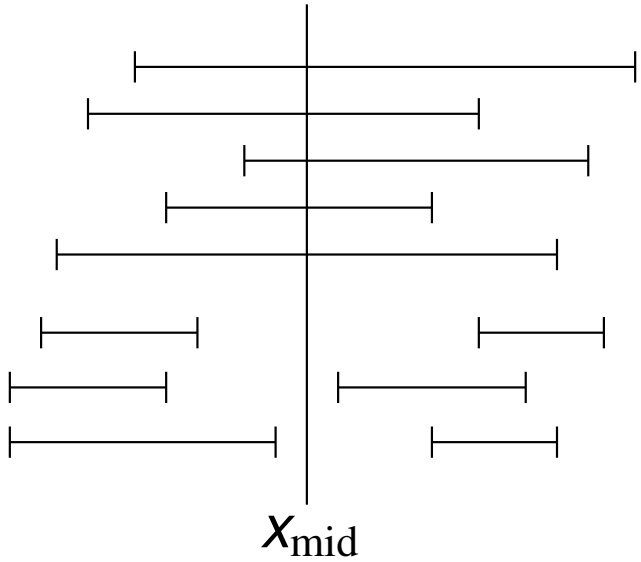




classification of intervals

Input: $S = \{[x_1 : x_1'], [x_2 : x_2'], \ldots, [x_n : x_n']\}$, a set of n intervals.

The median x_{mid} leads to a classification of the intervals in S.



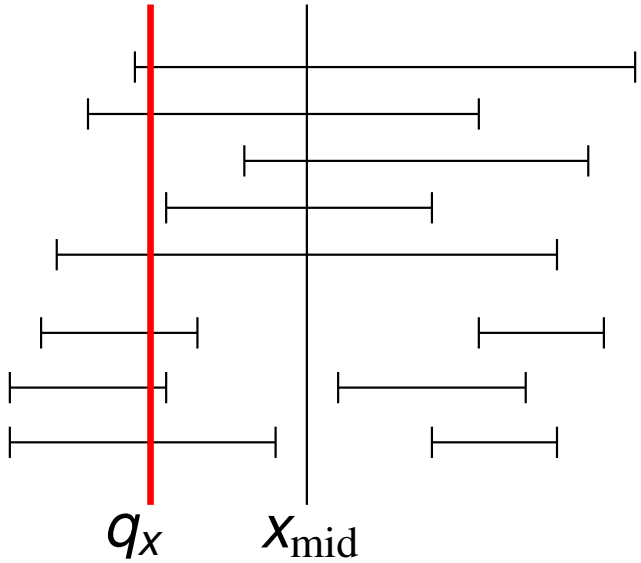
$$
\begin{aligned}
I_{\text{left}} &= \{\, [x : x'] \in S \mid x' < x_{\text{mid}} \,\} \\
I_{\text{mid}} &= \{\, [x : x'] \in S \mid x \leq x_{\text{mid}} \leq x' \,\} \\
I_{\text{right}} &= \{\, [x : x'] \in S \mid x_{\text{mid}} < x' \,\}
\end{aligned}
$$

query horizontal line segments

Query a set S of n horizontal line segments.

Input: (S, q_x), q_x represents the line $x = q_x$.

Output: $\{ s \in S \mid s \cap (x = q_x) \neq \emptyset \}$.



1. If $q_x = x_{\text{mid}}$, then report I_{mid}.
2. If $q_x < x_{\text{mid}}$, then report in I_{left}, I_{mid}.
3. If $q_x > x_{\text{mid}}$, then report in I_{right}, I_{mid}.

Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

the interval tree

Goal: report intervals that contain q_x in $O(\log(n) + k)$ time,
where k is the number of reported intervals.

$S = \{[x_1 : x_1'], [x_2 : x_2'], \ldots, [x_n : x_n']\}$, median x_{mid}.

$I_{\text{mid}} = \{\ [x : x'] \in S \mid x \leq x_{\text{mid}} \leq x'\ \}$

We sort $s \in S$ on left and right end points.

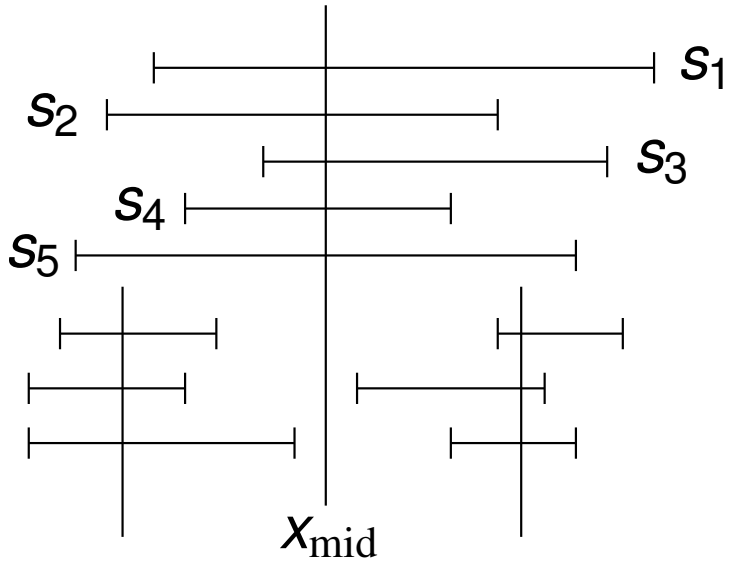
The set I_{mid} is stored twice:

1. $\mathcal{L}_{\text{left}}$ contains segments of I_{mid} sorted on the left end point,
2. $\mathcal{L}_{\text{right}}$ contains segments of I_{mid} sorted on the right end point.

a recursive structure

1. $\mathcal{L}_{\text{left}}$ contains segments of I_{mid} sorted on the left end point,
2. $\mathcal{L}_{\text{right}}$ contains segments of I_{mid} sorted on the right end point.

$I_{\text{mid}} = \{ s_1, s_2, s_3, s_4, s_5 \}$ is a set.
$\mathcal{L}_{\text{left}} = [s_5, s_2, s_1, s_4, s_3]$ and $\mathcal{L}_{\text{right}} = [s_4, s_2, s_5, s_3, s_1]$ are lists.



Exercise 1: On the example above, describe $\mathcal{L}_{\text{left}}$, $\mathcal{L}_{\text{right}}$ for I_{left}, I_{right}.

a recursive algorithm

Algorithm ConstructIntervalTree(S)

Input: a set S of intervals.
Output: the root of the interval tree which stores S.

1. if $S = \emptyset$ then
2. return an empty leaf

   else
3. $x_{\text{mid}} = \text{Median}(S)$
4. $[I_{\text{left}}, I_{\text{mid}}, I_{\text{right}}] = \text{Split}(S, x_{\text{mid}})$
5. $[\mathcal{L}_{\text{left}}, \mathcal{L}_{\text{right}}] = \text{Sort}(I_{\text{mid}})$
6. $v = \text{Node}(x_{\text{mid}}, \mathcal{L}_{\text{left}}, \mathcal{L}_{\text{right}})$
7. $\text{LeftChild}(v) = \text{ConstructIntervalTree}(I_{\text{left}})$
8. $\text{RightChild}(v) = \text{ConstructIntervalTree}(I_{\text{right}})$
9. return v

storage cost of an interval tree

Lemma (storage cost of an interval)

An interval tree for n horizontal line segments occupies $O(n)$ storage and has depth $O(\log(n))$.

- The depth $O(\log(n))$ follows from the median x_{mid}.

  The left and right child of the root hold at most $n/2$ intervals.

  The children of each node split the data of each node in half.

- A segment $[x : x']$ is stored only at the node where $x_{\text{mid}} \in [x : x']$, once in the list $\mathcal{L}_{\text{left}}$ and once in the list $\mathcal{L}_{\text{right}}$.

  The total number of nodes equals $2n$ and $2n$ is $O(n)$.

the cost of an interval tree

Lemma (cost of Algorithm ConstructIntervalTree)

An interval tree for n segments can be constructed in $O(n \log(n))$ time.

Consider the steps in the recursive algorithm:

3. $x_{\text{mid}} = \text{Median}(S)$
4. $[l_{\text{left}}, l_{\text{mid}}, l_{\text{right}}] = \text{Split}(S, x_{\text{mid}})$
5. $[\mathcal{L}_{\text{left}}, \mathcal{L}_{\text{right}}] = \text{Sort}(l_{\text{mid}})$

Then ConstructIntervalTree is applied to l_{left} and l_{right}.

Observe that the cost of Split is $O(n)$,
whereas the cost of Sort is $O(m \log(m))$, with m the size of l_{mid}.

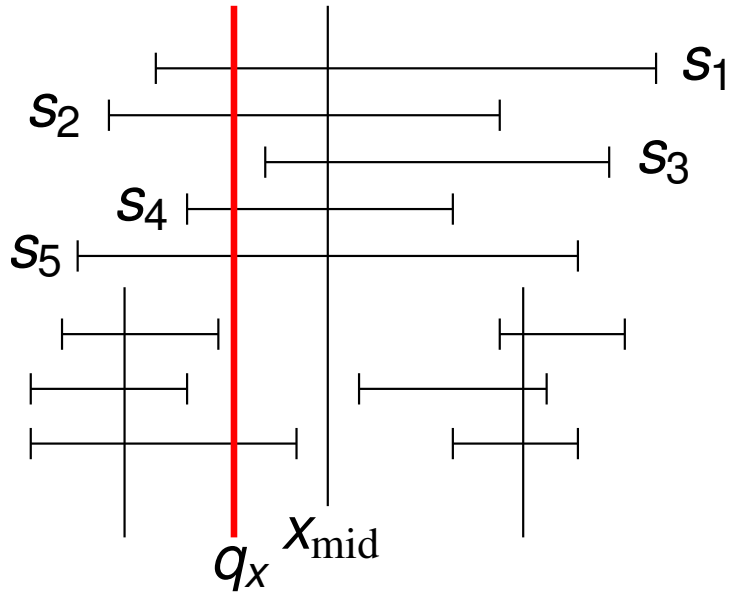
Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

query an interval tree

1. $\mathcal{L}_{\text{left}}$ contains segments of I_{mid} sorted on the left end point,
2. $\mathcal{L}_{\text{right}}$ contains segments of I_{mid} sorted on the right end point.

$I_{\text{mid}} = \{ s_1, s_2, s_3, s_4, s_5 \}$ is a set.
$\mathcal{L}_{\text{left}} = [s_5, s_2, s_1, s_4, s_3]$ and $\mathcal{L}_{\text{right}} = [s_4, s_2, s_5, s_3, s_1]$ are lists.



the algorithm to query an interval tree

Algorithm QUERYINTERVALTREE(v, q_x)

Input: root v of an interval tree and a query point q_x.

Output: all intervals that contain q_x.

1. if v is not a leaf then
2. if $q_x < x_{\text{mid}}(v)$ then
3. walk $\mathcal{L}_{\text{left}}(v)$, starting at interval with leftmost end point
4. in this walk, report all intervals that contain q_x
5. stop walking at the first interval that does not contain q_x
6. QUERYINTERVALTREE(LEFTCHILD(v), q_x)

else

9. walk $\mathcal{L}_{\text{right}}(v)$, starting at interval with rightmost end point
10. in this walk, report all intervals that contain q_x
11. stop walking at the first interval that does not contain q_x
12. QUERYINTERVALTREE(RIGHTCHILD(v), q_x)

cost of QUERYINTERVALTREE

What is the cost of QUERYINTERVALTREE?

- The depth of the interval tree is $O(\log(n))$.
- At node v, we report k_v intervals, in time $O(k_v)$.

So the total time is $O(\log(n) + k)$, where $k = \sum_{\text{node } v} k_v$.

Theorem (cost of interval trees)

Given n intervals, an interval tree

- *uses $O(n)$ storage and takes $O(n \log(n))$ time to construct; and*
- *a query takes $O(\log(n) + k)$ time, where k is the size of the output.*

Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

running an example of CGAL

On a Window Subsystem for Linux (WSL), running Ubuntu:

1. Run `sudo apt-get install` followed by `libgmp-dev`, `libmpfr-dev`, `libcgal-dev`.
2. Download `CGAL-5.6-examples.zip`.
3. Compile `intervals.cpp` from the `Interval_skip_list` folder.

The `intervals.cpp` constructs an interval tree for given numbers n and d, inserting n intervals $[i, i+d]$. It then executes queries, writes the results, and removes the intervals.

Exercise 2: Run `intervals.cpp` for sufficiently large values for n and d. Report the observed times for the construction of the data structure. Do you observe the $O(n \log(n))$ as n grows?

Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

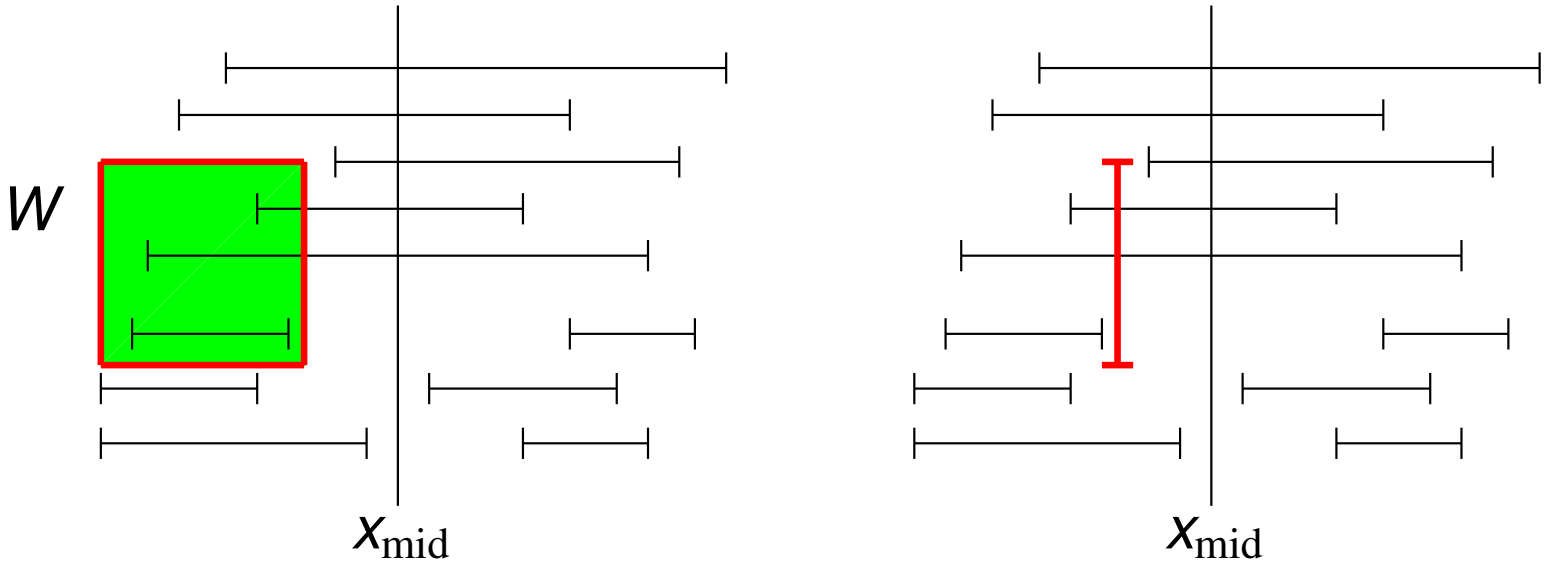
a rectangular query window

Query a set S of n horizontal line segments.

Input: S and the query window $W = [q_x : q'_x] \times [q_y : q'_y]$.

Output: $\{ s \in S \mid s \cap W \neq \emptyset \}$.

Consider replacing W on the left by $q'_x \times [q_y : q'_y]$ on the right:

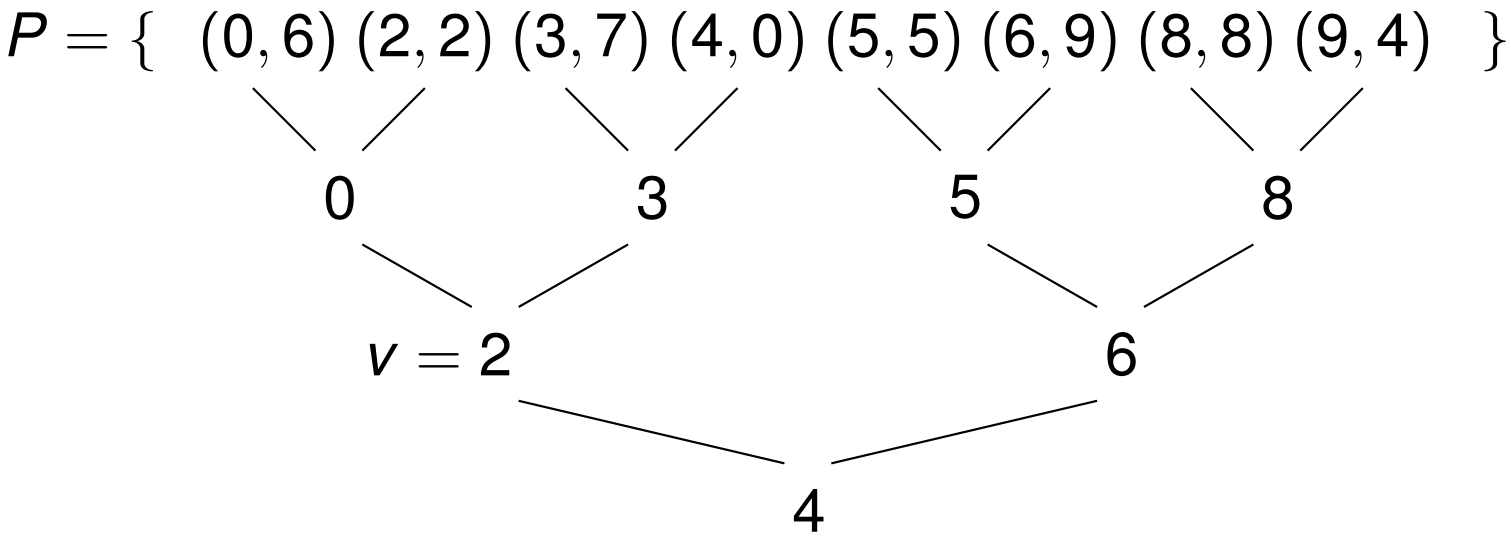


Exercise 3: What happens when we run Algorithm QUERYINTERVALTREE on the example shown above?

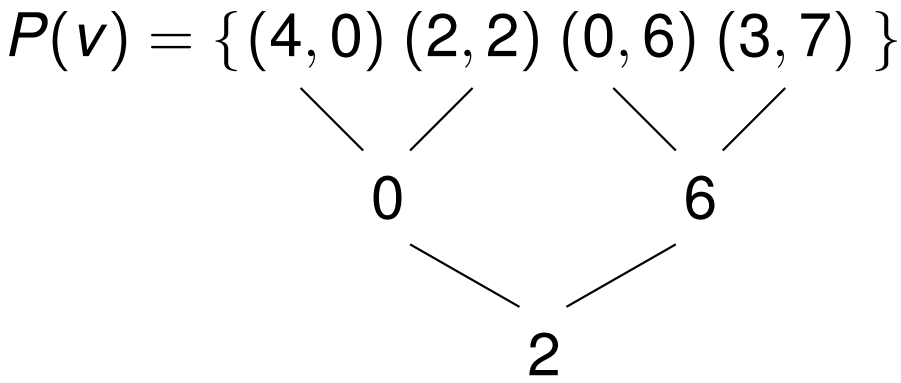
Interval Trees

1. Windowing Queries
   - zoom in on a window
   - application of range queries
2. The Interval Tree
   - a data structure for a simplified problem
   - storing intervals in a tree
   - query an interval tree
   - running an example of CGAL
3. Rectangular Query Windows
   - horizontal line segments in a window
   - application of range trees

a two dimensional range tree



The tree $T_{\text{assoc}}(v)$ associated to $v = 2$:



application of range trees

In the interval tree, instead of the sorted lists $\mathcal{L}_{\text{left}}$ and $\mathcal{L}_{\text{right}}$, we have two range trees at each node v, $\mathcal{T}_{\text{left}}$ and $\mathcal{T}_{\text{right}}$:

- $\mathcal{T}_{\text{left}}$ stores the left end points of the segments in $I_{\text{mid}}(v)$,
- $\mathcal{T}_{\text{right}}$ stores the right end points of the segments in $I_{\text{mid}}(v)$.

Applying results on range trees, we obtain:

1. construction takes $O(n \log(n))$ time,
2. storage cost is $O(n \log(n))$, and
3. query time is $O(\log^2(n) + k)$, where k is the output size.

Exercise 4: The horizontal line segments on slide 25 are defined by $S = \{((0,0),(17,0)), ((27,0),(35,0)), ((5,21),(30,21)), ((8,24),(40,24)), ((3,12),(35,12)), ((10,15),(27,15)), ((30,7),(38,7)), ((15,18),(37,18)), ((2,7),(12,7)), ((21,3),(33,3)), ((0,3),(10,3))\}$. Construct a two dimensional range tree to store S.

summary and exercises

The problem of zooming in on n horizontal line segments has construction and storage cost of $O(n \log(n))$, with query cost $O(\log^2(n) + k)$, where k is the output size.

We covered section 10.1 in the textbook.

Consider the following activities, listed below.

1. Write the solutions to exercises 1, 2, 3, and 4.
2. Read the CGAL documentation on the Interval Skip List.
3. Consider the exercises 1, 4, 9 in the textbook.